

Dear Enemy, Ticket to Ride is NOT Just an Adjacency Matrix

Refracted Light System[s Incorporated] (they/them)

Abstract

ANTI-DEDICATED TO ALL ENEMIES OF THE REFRACTED LIGHT CEO BUT ESPECIALLY THE ONE WHO SAID "TICKET TO RIDE IS JUST AN ADJACENCY MATRIX"

In this paper, we present an algorithm for connecting Ticket to Ride *Destination Tickets* (or solve the "n-pairs minimum forest problem" if you're a nerd) with humorous and petty commentary because otherwise no one would care about the cool algorithm we made.

1 Introduction

Ok, so we are a little petty with the anti-dedication (call that a degradation? Disdain-agation?) to one of our many enemies who will not be named but know who they are, for they chose to be like that. However, we made a cool thing, so suck it.

The cool thing we made connects sets of Ticket to Ride (a popular board game) *Destination Tickets* (which are pairs of cities in Ticket to Ride that must be connected by at least one path of claimed adjacent city routes in order to get points) in a manner that requires the minimum number of turns or trains. Equivalently, this solves the n-pairs minimum forest problem (where pairs of vertices must be connected by at least one path in such a manner that the total sum of the taken edge weights is its minimum value). I made up this problem, so don't bother looking it up because it's a useless problem for everything except Ticket to Ride.

2 Algorithm

The algorithm we made requires all the single pair shortest paths, which I have reproduced an algorithm for below. We had to independently come up with this Dijkstra-based algorithm because Google search sucks nowadays, but I know we're not the first to need it. Be prepared for the sloppiest algorithm you've ever seen:

G : initial graph

v_{start} : start vertex

v_{end} : end vertex

Returns two values:

the weight of the shortest paths,

and a list of the shortest paths,
which are in turn a set of edges.

```

BRANCHEDDIJKSTRA( $G, v_{start}, v_{end}$ )
  root  $\leftarrow$  TreeNode( $[0, v_{start}]$ , 0)
   $t \leftarrow$  Tree(root)
  branches  $\leftarrow$  1
  visited  $\leftarrow \emptyset$ 
  distances  $\leftarrow \emptyset$ 
  for each vertex  $v_i$  in vertices( $G$ ):
    visited[ $v_i$ ]  $\leftarrow$  false
    distances[ $v_i$ ]  $\leftarrow \infty$ 
  distances[ $v_{start}$ ]  $\leftarrow$  0
  for each vertex in vertices( $G$ ):
    minDist  $\leftarrow \infty$ 
     $v_{minDist} \leftarrow \emptyset$ 
    for each tuple  $(v_d, d)$  in pairs(distances):
      if  $d < \text{minDist} \wedge \neg \text{visited}[v_d]$ :
         $v_{minDist} \leftarrow v_d$ 
        minDist  $\leftarrow d$ 
    if  $v_{minDist} = \emptyset$ 
      return  $\emptyset, \emptyset$ 
    for each vertex  $v_j$  in vertices( $G$ ):
      if  $G$  has edge( $v_j, v_{minDist}$ ):
        if visited[ $v_j$ ]:
          continue
         $s \leftarrow$  from  $G$ , get edge( $v_j, v_{minDist}$ )
         $d_{new} \leftarrow$  distances[ $v_{minDist}$ ] + weight( $s$ )
         $d_j \leftarrow$  distances[ $v_j$ ]
        if  $d_j > d_{new}$ :
          distances[ $v_j$ ]  $\leftarrow d_{new}$ 
        if  $d_j \geq d_{new}$ :
           $t_{copy} \leftarrow \text{deepCopy}(t)$ 
          newBranches  $\leftarrow$  0
          found  $\leftarrow$  false
          for  $k$  in range(0, branches):
            if  $t$  has key( $[k, v_j]$ ):
              for  $l$  in range(0, branches):
                if  $t$  has key( $[l, v_{minDist}]$ ):
                   $n_0 \leftarrow$  from  $t$  get node by key( $[l, v_{minDist}]$ )
                  if value( $n_0$ ) > distances[ $v_{minDist}$ ]:
                    from  $t_{copy}$ , remove node by key( $[l, v_{minDist}]$ )
                else:
                  to  $t_{copy}, \dots$ 
                  add child TreeNode( $[\text{branches} + \text{newBranches}, v_j]$ ,  $d_{new}$ )...
                  with parent node key  $[l, v_{minDist}]$ 

```

```

        newBranches ← newBranches + 1
    found ← true
    break
if ¬found:
    for k in range(0,branches):
        if t has key([k, vminDist]):
            n1 ← from t get node by key([k, vminDist])
            if value(n1) > distances[vminDist]:
                from tcopy, remove node by key([k, vminDist])
            else:
                to tcopy, add child TreeNode([k, vj], dnew)...
                    with parent node key [k, vminDist]
        branches ← branches + newBranches
    t ← tcopy
if weight(s) = 0:
    tcopy ← deepCopy(t)
    newBranches ← 0
    for k in range(0,branches):
        if t has key([k, vj]) ∧ ¬(t has key([branches + newBranches, vminDist])):
            n2 ← from t get node by key([k, vj])
            if ¬key(parent(n2)) = vminDist:
                to tcopy, add child TreeNode(key(n2), value(n2))...
                    with parent node key [branches + newBranches, vminDist]
        branches ← branches + newBranches
    t ← tcopy
visited[vminDist] ← true
if ¬(values(visited) contains false):
    retPaths ← ∅
    for k in range(0, branches):
        if t has node with key([k, vend]):
            path ← from t, get path to node with key([k, vend])
            add path to retPaths
    remove duplicates from retPaths
    return distances[vend], retPaths
return ∅, ∅

```

Wow! That's ugly! Yeah, I probably didn't need to make that so detailed for a SIGBOVIK paper.

The next algorithm, my helper function MWFHelp, is kinda like a meta version of the previous algorithm, with the addition of recursion. Every CS major just shuddered when reading that, right? But it's not *that* bad. I mean, its worst-case running time is something like $O(\frac{V^2 E^{n+1}}{(E-1)^2})$ which is TERRIBLE for large inputs, but it's functional for Ticket to Ride on modern computers. At this point in writing the paper, I went down a rabbithole trying to see how many milliseconds it takes on my computer when compiled to native code, which is harder than it sounds because Java.awt is not very well supported by GraalVM. Anyway, the answer is that if I give it a medium problem (5 destination tickets) it will take approximately 17 milliseconds, a hard problem (10 destination

tickets) 100-150 milliseconds, and the hardest problem my program supports (16 destination tickets) takes 3-4 or 7-8 seconds depending on whether the window is focused or not. What was I talking about? Oh yeah, our algorithm.

G_0 : initial graph

P_0 : list of vertex pairs

P_{done} : map of vertex pairs to booleans,

indicating which vertex pairs have been fully processed by MWFHELP

E_{taken} : a set of previously taken edges

w_0 : total weight of previously taken edges

Returns a list of results.

Each result has a graph with the weights of taken edges set to 0, the taken edges, and the total weight of the taken edges.

If some pairs cannot be connected,

this algorithm's results are based on the pairs that can.

MWFHELP($G_0, P_0, P_{done}, E_{taken}, w_0$)

if $\neg P_{done}$ contains false:

return $[(G_0, E_{taken}, w_0)]$

$p_{min} \leftarrow \emptyset$

$w_{min} \leftarrow \infty$

$S_{min} \leftarrow \emptyset$

for pair p_1 in P_0 :

if $\neg P_{done}[p]$:

$w_1, S_1 \leftarrow \text{BRANCHEDDIJKSTRA}(G, p[0], p[1])$

if $w_1 \neq \emptyset \wedge w_1 < w_{p,min}$:

$w_{min} \leftarrow w_1$

$S_{min} \leftarrow S_1$

$p_{min} \leftarrow p_1$

if $p_{min} = \emptyset$:

COMMENT: Impossible to connect remaining pairs

return $[(G_0, E_{taken}, w)]$

$w_2 \leftarrow w_0 + w_{min}$

$A_{min} \leftarrow \emptyset$

$P_{done}[p_{min}] \leftarrow \text{true}$

for set of edges s in S_{min} :

$G_1 \leftarrow \text{copy}(G_0)$

$E_1 \leftarrow \text{copy}(E_{taken})$

to E_1 add all of (s)

from E_1 remove weight 0 edges

for edge e in E_1 :

in G_1 set weight of e to 0

$A_1 \leftarrow \text{MWFHELP}(G_1, P_0, \text{copy}(P_{done}), E_1, w_2)$

if $A_{min} = \emptyset \wedge A_{min}[0][2] < A_1[0][2]$:

$A_{min} \leftarrow A_1$

else if $A_{min}[0][2] = A_1[0][2]$:

```

        to  $A_{min}$  add all of  $(A_1)$ 
    return  $A_{min}$ 

```

See? Not *that* bad.

The final algorithm is just MWFHELP with some initial parameters. I won't bother writing it properly because this is a SIGBOVIK paper; initialize P_{done} to false for each pair in P_0 , E_{taken} should be an empty set, and $w_0 \leftarrow 0$. That leaves you with two parameters to play with as you like.

3 Implementation Details

The authors used the Java programming language to implement the cool thing.

3.1 Edges and Vertices of Graphs

The generic type of vertices, v , was implemented by objects implementing the `Comparable` interface.

The generic type of edge weights, w , was implemented by objects extending the `Number` abstract class.

Thus the generic type of edges was implemented as a class `Edge<v extends Comparable<v>, w extends Number>`, with three fields: `v start`, `v end`, and `w weight`. `start` must be less than `end` to avoid bugs. `start` and `end` represent the starting and ending vertices of an edge respectively. `weight` must be non-negative, as it represents the weight of an edge.

3.2 Graphs

Generic graphs were implemented as a class `Graph<v extends Comparable<V>, w extends Number>` with `v` and `w` retaining their definitions in section 3.1. Graphs have two fields: `TreeMap<v, ArrayList<Edge<v,w>>> adjacencies` and `w val0`. The core of the class is in `adjacencies`, as it maps each vertex to the edges that contain it. `val0` is `w`'s zero value, and is only there because it was harder to check if both floating-point numbers and integer `ws` were zero without it. Thinking about it now, we could have probably gotten away with type casting. Also, we probably should've implemented graphs as a 2D array for constant time adjacency checking. Oh well.

3.3 Trees and Tree Nodes

Generic tree nodes were implemented as a class `TreeNode<K extends Comparable<K>, V>` where `K` is the key type and `V` is the node value type. It has three fields: `K id`, `V value`, and `K parent`. `id` is the node id, `value` is the node value, and `parent` is the id of the node's parent.

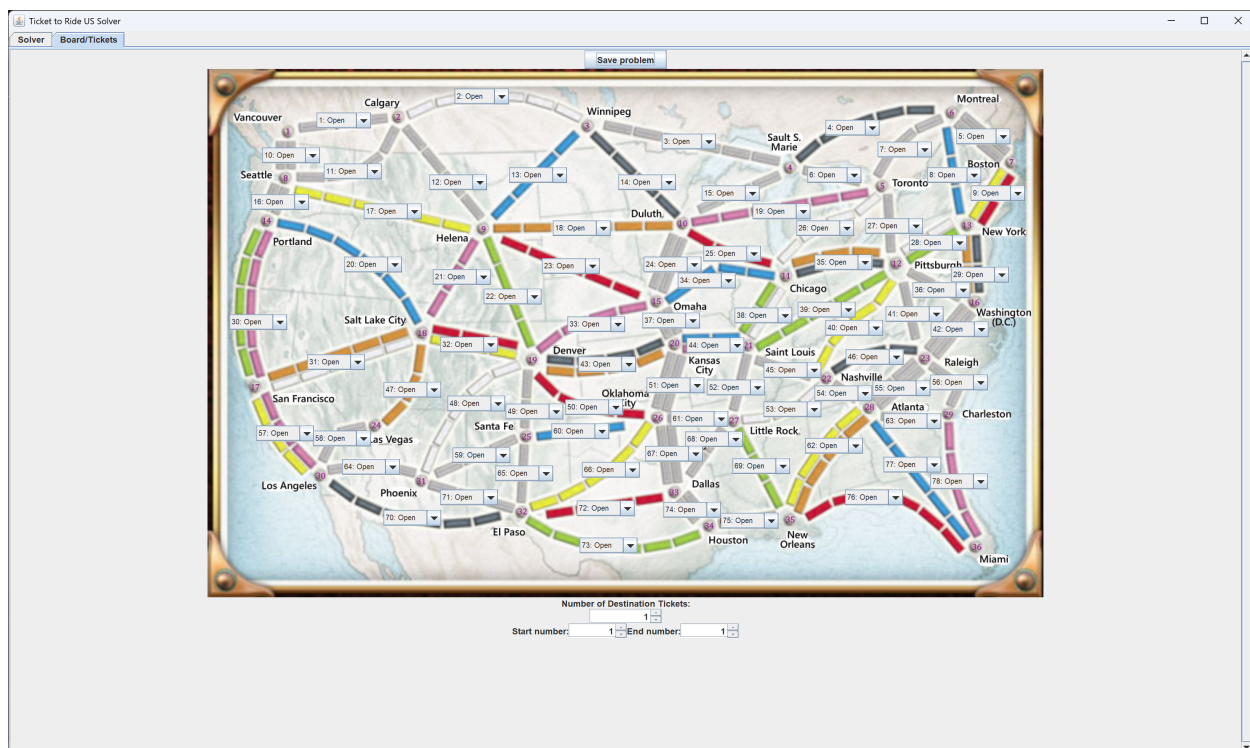
Generic trees were implemented as a class `Tree<K extends Comparable<K>, V>` where `K` and `V` are as in `TreeNode`. It contains two fields: `TreeSet<TreeNode<K,V>> allNodes` and `TreeNode<K,V>`

`root.allNodes` is the set (implemented by `java.util.TreeSet`) of all nodes in the tree. `root` is the root node of the tree.

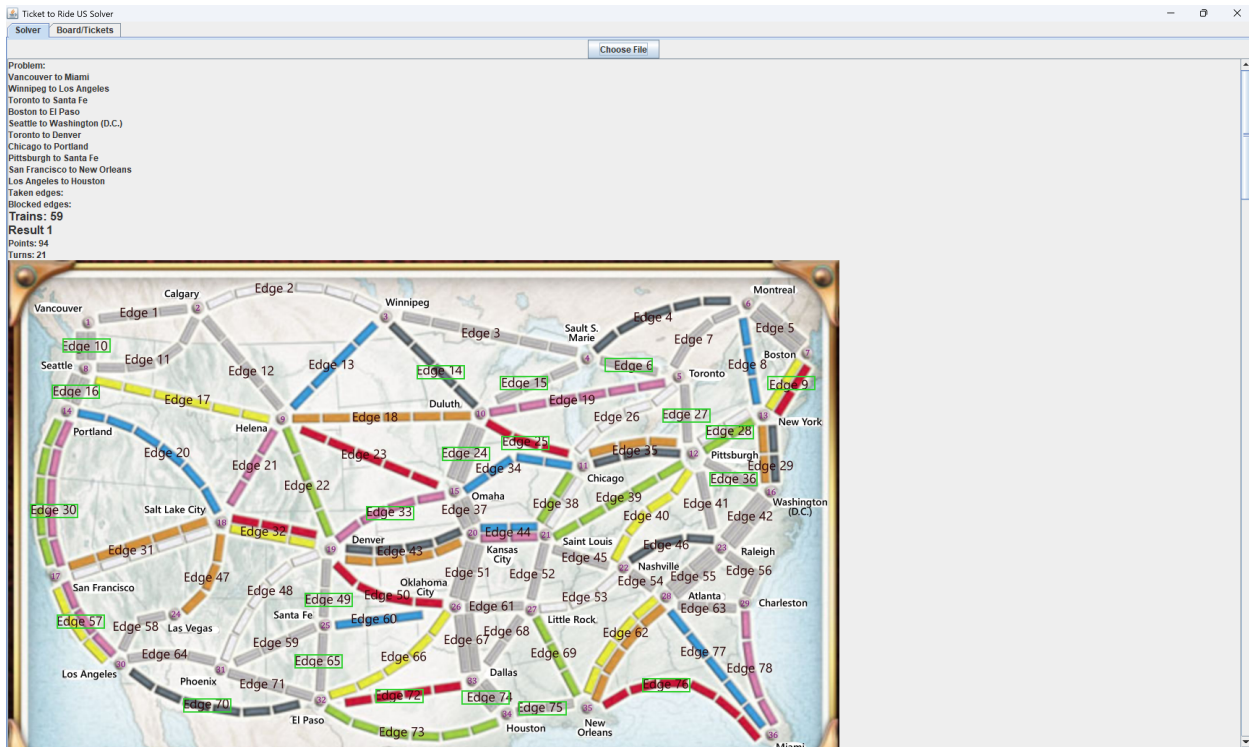
3.4 GUI Pictures

I am sorry for our ugly GUI, but it has one :)

The below screenshot shows the Board/Tickets tab. It has an annotated Ticket to Ride board with dropdown menus on edges that can be open, taken, or blocked. It also has a destination ticket input, where you select the number of destination tickets (1-16) and their start and end numbers on the board.



The below screenshots show the Solver tab. It describes the problem solved and then gives you the amount of trains it takes, then shows each result organized by descending number of points. The results are shown via outlining the edge annotations in a really ugly green that we chose because it was so visible and ugly.



4 You Don't Want Me, Just My Code

If you want our code or have any questions, contact lux@theweirdwizards.com